Լաբորատոր աշխատանք 10

Դինամիկ անդամեր և լռելյայն կոնստրուկտոր

Աշխատանքի նպատակը: Ստեղծել դատարկ և տվյալներով օբյեկտներ, անվտանգ փոփոխել դինամիկ զանգվածի չափը։

Տեսական մաս: Լռելյայն կոնստրուկտորը կանչվում է առանց արգումենտների։ Դատարկ օբյեկտում n=0 և data=nullptr արժեքները նկարագրում են վավեր վիճակ։ Չափը փոխելիս նախ հատկացնել նոր զանգված, փոխանցել տարրերը և միայն դրանից հետո ազատել հինը։

Ընդհանուր պահանջներ:

1. Դասում պահել private double* զանգված, n չափ և p պարամետր։ Լռելյայն կոնստրուկտորը ստեղծում է դատարկ օբյեկտ՝ n=0, data=nullptr և պայմանում նշված p-ով։ Ավելացնել պարամետրերով կոնստրուկտոր, դեստրուկտոր, append(value), removeAt(index), print() և երեք հաշվարկները։ Չափի փոփոխությունը կատարել նոր զանգված հատկացնելով, տվյալները պատճենելով և հին հիշողությունն ազատելով։ Պատճենում և օբյեկտների վերագրում չկիրառել։

2. Տվյալների ծավալ։ Սկզբնական ոչ դատարկ հավաքածուի չափը $3 \le n \le 30$ է։ Չափի փոփոխություն պահանջող մեթոդներում թույլատրվում է $0 \le n \le 30$։ Բոլոր թվերը վերջավոր են։ Հավասար լավագույն արդյունքների դեպքում ընտրել առաջին ինդեքսը։ Իրական թվերի հավասարությունը համեմատել 10^{-9} ճշտությամբ։

3. Դատարկ վիճակ։ Գումարները և քանակները զրո են, լավագույն տարրի ինդեքսը՝ -1։ Միջինի կամ հարևանների տարբերության համար բավարար տարրեր չլինելու դեպքում արտածել «Տվյալները բավարար չեն»։ Բազմանդամի դատարկ գործակիցները նշանակում են զրոյական բազմանդամ։ Ինտեգրալի համար պահանջվում է առնվազն երկու արժեք։

4. Վավերացում։ Սխալ չափով կամ պարամետրերով օբյեկտ չստեղծել. մուտքը կրկնել main()-ում։ Թույլատրելիության կանոնները ստուգել նաև փոփոխող մեթոդներում։ Սխալ ինդեքսի կամ արժեքի դեպքում վերադարձնել false և պահպանել նախկին տվյալները։ Հաշվարկները չպետք է փոփոխեն օբյեկտը։

Առաջադրանք: Ածանցյալի նշանի ուսումնասիրություն

Տվյալներ։ Ստեղծել DerivativeStudy դաս՝ պահելով x կետերի զանգվածը և p գործակիցը։ $f(x)=x^3+p\cdot x$, իսկ $f'(x)=3x^2+p$։

Հաշվարկներ։ Հաշվել ածանցյալների միջինը, գտնել բացարձակ արժեքով ամենափոքր ածանցյալի ինդեքսը և բացասական ածանցյալների քանակը։

Թույլատրելի արժեքներ։ x և p արժեքները վերջավոր իրական թվեր են։

Իրականացում։Լռելյայն p=0։ Դատարկ օբյեկտում append-ով ավելացնել օրինակի զանգվածի տարրերը՝ պահելով լռելյայն p-ն։ Առանձին օբյեկտը ստեղծել օրինակի p-ով։ Առաջինից հեռացնել ինդեքս 1-ի տարրը և երկու օբյեկտների համար հաշվել արդյունքները։

Փորձնական տվյալներ։ x={-2,-1,0,2}, p=-3։ Ինդեքսները սկսվում են 0-ից։

```
#include <iostream>
#include <cmath>
#include <limits>
#include <stdexcept>
using namespace std;
class DerivativeStudy {
private:
    double* data;
    int n;
    double p;
public:
    DerivativeStudy() {
        data = nullptr;
        n = 0;
        p = 0 }
    DerivativeStudy(double arr[], int size, double parameter) {
        data = nullptr;
        n = 0;
        p = 0;
        if (arr == nullptr || size < 3 || size > 30 ||
            !isfinite(parameter)) {
```

```
        throw invalid_argument("Invalid data"); }
    for (int i = 0; i < size; i++) {
        if (!isfinite(arr[i])) {
            throw invalid_argument("Invalid array value");}}
    data = new double[size];
    n = size;
    p = parameter;
    for (int i = 0; i < n; i++) {
        data[i] = arr[i];}}
DerivativeStudy(const DerivativeStudy&) = delete;
DerivativeStudy& operator=(const DerivativeStudy&) = delete;
~DerivativeStudy() {
    delete[] data;}
bool append(double value) {
    if (!isfinite(value) || n >= 30) {
        return false;}
    double* temp = new double[n + 1];
    for (int i = 0; i < n; i++) {
        temp[i] = data[i];}
    temp[n] = value;
    delete[] data;
    data = temp;
    n++;
    return true;}
bool removeAt(int index) {
    if (index < 0 || index >= n) {
        return false;}
    double* temp = nullptr;
    if (n > 1) {
        temp = new double[n - 1];
```

```
        for (int i = 0, j = 0; i < n; i++) {
            if (i != index) {
                temp[j++] = data[i];}}}
    delete[] data;
    data = temp;
    n--;
    return true;}
double derivative(int index) const {
    return 3 * data[index] * data[index] + p;}
    void print() const {
    cout << "p = " << p << endl;
    cout << "X values: ";
    for (int i = 0; i < n; i++) {
        cout << data[i] << " ";}
    cout << endl << "Derivatives: ";
    for (int i = 0; i < n; i++) {
        cout << derivative(i) << " ";}
    cout << endl;}
void averageDerivative() const {
    if (n == 0) {
        cout << "Not enough data" << endl;
        return;}
    double sum = 0;
    for (int i = 0; i < n; i++) {
        sum += derivative(i);}
    cout << "Average derivative: " << sum / n << endl;}
int minAbsIndex() const {
    if (n == 0) {
        return -1;}
    int index = 0;
```

```
        double minValue = fabs(derivative(0));
        for (int i = 1; i < n; i++) {
            double current = fabs(derivative(i));
            if (current < minValue - 1e-9) {
                minValue = current;
                index = i;}}
        return index;}
    int negativeCount() const {
        int count = 0;
        for (int i = 0; i < n; i++) {
            if (derivative(i) < -1e-9) {
                count++;}}
        return count;}};
int main() {
    int size;
    double x[30];
    double p;
    do {
        cout << "Enter array size (3-30): ";
        if (cin >> size && size >= 3 && size <= 30) {
            break;}
        cout << "Invalid size. Try again." << endl;
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    } while (true);
    for (int i = 0; i < size; i++) {
        do {
            cout << "Enter x[" << i << "]: ";
            if (cin >> x[i] && isfinite(x[i])) {
                break;}
```

```
        cout << "Invalid value. Try again." << endl;
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    } while (true);}
do {
    cout << "Enter p: ";
    if (cin >> p && isfinite(p)) {
        break;}
    cout << "Invalid value. Try again." << endl;
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
} while (true);
try {
    DerivativeStudy first;
    for (int i = 0; i < size; i++) {
        first.append(x[i]);}
    DerivativeStudy second(x, size, p);
    first.removeAt(1);
    cout << endl << "FIRST OBJECT:" << endl;
    first.print();
    first.averageDerivative();
    cout << "Minimum absolute derivative index: "
        << first.minAbsIndex() << endl;
    cout << "Negative derivatives count: "
        << first.negativeCount() << endl;
    cout << endl << "SECOND OBJECT:" << endl;
    second.print();
    second.averageDerivative();
    cout << "Minimum absolute derivative index: "
        << second.minAbsIndex() << endl;
```

```
    cout << "Negative derivatives count: "
         << second.negativeCount() << endl;}
catch (const exception& e) {
    cout << "Error: " << e.what() << endl;}
return 0;}
```